

Lab: From Compose to Production — Docker Swarm Orchestration

1. The two project structures

You maintain **two separate trees**. Conflating them is the #1 structural mistake teams make.

1a. Source code tree — `safa-shop/` (what you `docker build`)

```
safa-shop/
├── backend/                                # builds image  safa/api:1.0
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app/
│       ├── __init__.py
│       └── main.py
└── frontend/                              # builds image  safa/nginx:1.0 (static site
+ reverse proxy)
    ├── Dockerfile
    └── html/
        └── index.html
```

The `frontend/` folder is the **static site**. Because you chose "nginx serves the frontend," these assets are baked into the nginx image at build time — there is no separate `frontend` service. The proxy behaviour (`nginx.conf`) is delivered separately, at deploy time, as a Swarm **config** (see the deployment tree). Image carries the *content*; the stack carries the *behaviour*.

1b. Deployment tree — `safa-swarm-deploy/` (what you `docker stack deploy`)

```
safa-swarm-deploy/
├── .gitignore
└── postgres/
```

```

|   ├── docker-compose.yaml
|   ├── secrets/
|   |   ├── postgres_user.txt
|   |   ├── postgres_password.txt
|   |   └── postgres_db.txt
|   └── init/
|       └── 001_init.sql
└── redis/
    ├── docker-compose.yaml
    └── secrets/
        └── redis_password.txt
└── backend/
    ├── docker-compose.yaml      # self-contained: declares its OWN copies of
the creds it needs
    └── secrets/
        ├── postgres_user.txt
        ├── postgres_password.txt
        ├── postgres_db.txt
        └── redis_password.txt
└── nginx/
    ├── docker-compose.yaml
    └── config/
        └── nginx.conf          # delivered as a Swarm config, not a bind
mount
└── deploy.sh                  # convenience: the one merged `stack deploy`
command

```

Self-contained rule (important, used throughout): because `deploy.sh` deploys each service as its **own** `docker stack deploy` command (not one merged `-c ... -c ...`), every compose file must stand on its own — each secret a service references is declared **top-level in that same file**. Postgres and Redis hold the canonical credential files. The backend keeps **its own copies** and declares them top-level too, mounting each at the path the app expects (`/run/secrets/postgres_user`, ...) via a `source`/`target` pair, so `backend/docker-compose.yaml` can be deployed by itself without depending on the other files being merged in. To keep the api's objects distinct from Postgres/Redis's (and dodge the "secret already exists / immutable" trap), the backend gives its top-level secrets `api_`-prefixed names — the `target:` still lands them at the unprefixed paths the app reads. (Prefer not to duplicate the material? See the **external** alternative in §5c.)

Create both trees now:

```
mkdir -p ~/safa-shop/backend/app ~/safa-shop/frontend/html
mkdir -p ~/safa-swarm-deploy/postgres/{secrets,init} \
~/safa-swarm-deploy/redis/secrets \
~/safa-swarm-deploy/backend/secrets \
~/safa-swarm-deploy/nginx/config
```

```
# (no output on success – verify with `tree` or `find`)
```

```
find ~/safa-shop ~/safa-swarm-deploy -type d | sort
```

```
/home/ubuntu/safa-shop
/home/ubuntu/safa-shop/backend
/home/ubuntu/safa-shop/backend/app
/home/ubuntu/safa-shop/frontend
/home/ubuntu/safa-shop/frontend/html
/home/ubuntu/safa-swarm-deploy
/home/ubuntu/safa-swarm-deploy/backend
/home/ubuntu/safa-swarm-deploy/backend/secrets
/home/ubuntu/safa-swarm-deploy/nginx
/home/ubuntu/safa-swarm-deploy/nginx/config
/home/ubuntu/safa-swarm-deploy/postgres
/home/ubuntu/safa-swarm-deploy/postgres/init
/home/ubuntu/safa-swarm-deploy/postgres/secrets
/home/ubuntu/safa-swarm-deploy/redis
/home/ubuntu/safa-swarm-deploy/redis/secrets
```

2. Initialize the Swarm

The VM is a plain Docker host right now. Confirm, then promote it to a single-node swarm.

```
docker info --format '{{.Swarm.LocalNodeState}}'
```

```
Swarm: inactive
```

inactive = not a swarm yet. Initialize it:

```
docker swarm init
```

```
Swarm initialized: current node (77fzoq1ktcg0e2o1zt7gq2uz2) is now a manager.
```

To add a worker to this swarm, run the following command:

```
docker swarm join --token SWMTKN-1-5c7lis3bsqunpkjoto6ksxx12oatbt655p4qb4ni16l4b58x5z-4mf6friqwybqxa0bs5ukw321o 10.10.10.13:2377
```

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.

What just happened: this single VM is now a Swarm **manager** (it holds the Raft state) *and* a **worker** (it can run tasks). On a one-node swarm both roles live on the same box — perfect for this lab. The **join** token is how you'd add more VMs later; you don't need it today.

If **docker swarm init** complains **could not choose an IP address to advertise since this system has multiple addresses**, pin one explicitly: **docker swarm init -advertise-addr <your-VM-IP>**.

Verify the node is **Ready** and **Active**:

```
docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
r37lo2sdi9blawiy282djt5uw *	swarm-01	Ready	Active	Leader	29.3.1

The ***** marks the node you're issuing commands from; **Leader** means it's the active Raft manager.

3. Write the application source (**safa-shop/**)

3a. **backend/** — the FastAPI API, adapted to read secrets from files

~/safa-shop/backend/requirements.txt:

```
fastapi==0.115.0
uvicorn[standard]==0.30.6
asynccpg==0.29.0
redis==5.0.8
```

~/safa-shop/backend/app/__init__.py — empty file:

```
: > ~/safa-shop/backend/app/__init__.py
```

```
~/safer-shop/backend/app/main.py:
```

```
import json
import os
from contextlib import asynccontextmanager

import asyncpg
import redis.asyncio as redis
from fastapi import FastAPI, HTTPException

def read_secret(name: str, default: str | None = None) -> str | None:
    """Return the value of NAME_FILE (a Docker secret) if set, else the
    plain env var.

    Swarm mounts secrets as files under /run/secrets/. The official postgres
    and
    redis images use the same *_FILE convention; we mirror it here so the
    SAME
    app image runs unchanged under Compose (env vars) or Swarm (secret
    files).
    """
    file_path = os.environ.get(f"{name}_FILE")
    if file_path and os.path.exists(file_path):
        with open(file_path, encoding="utf-8") as fh:
            return fh.read().strip() # .strip() defuses the
trailing-newline bug
    return os.environ.get(name, default)

POSTGRES_USER = read_secret("POSTGRES_USER")
POSTGRES_PASSWORD = read_secret("POSTGRES_PASSWORD")
POSTGRES_DB = read_secret("POSTGRES_DB")
POSTGRES_HOST = os.environ.get("POSTGRES_HOST", "postgres")

REDIS_PASSWORD = read_secret("REDIS_PASSWORD")
REDIS_HOST = os.environ.get("REDIS_HOST", "redis")

DATABASE_URL = (
    f"postgresql://{POSTGRES_USER}:{POSTGRES_PASSWORD}"
    f"@{POSTGRES_HOST}:5432/{POSTGRES_DB}"
)
REDIS_URL = f"redis://{REDIS_PASSWORD}@{REDIS_HOST}:6379/0"
```

```

@asynccontextmanager
async def lifespan(app: FastAPI):
    # In Swarm there is no depends_on: postgres/redis may not be ready when
    this
    # container starts. If create_pool fails, the process exits non-zero and
    the
    # service's restart_policy brings it back – that retry loop IS the
    ordering.
    app.state.pg_pool = await asyncpg.create_pool(DATABASE_URL, min_size=2,
max_size=10)
    app.state.redis = redis.from_url(REDIS_URL, decode_responses=True)
    yield
    await app.state.pg_pool.close()
    await app.state.redis.aclose()

app = FastAPI(lifespan=lifespan)

@app.get("/health")
async def health():
    async with app.state.pg_pool.acquire() as conn:
        await conn.fetchval("SELECT 1")
    await app.state.redis.ping()
    return {"status": "ok"}

@app.get("/api/products/{product_id}")
async def get_product(product_id: int):
    cache_key = f"product:{product_id}"
    cached = await app.state.redis.get(cache_key)
    if cached:
        payload = json.loads(cached)
        payload["source"] = "cache"
        return payload

    async with app.state.pg_pool.acquire() as conn:
        row = await conn.fetchrow(
            "SELECT id, name, price_cents FROM products WHERE id = $1",
product_id
        )

```

```

if row is None:
    raise HTTPException(status_code=404, detail="product not found")

payload = dict(row)
await app.state.redis.set(cache_key, json.dumps(payload), ex=60)
payload["source"] = "postgres"
return payload

```

~/safer-shop/backend/Dockerfile:

```

FROM python:3.12-slim AS base

# curl is used by the container's own healthcheck (see backend/docker-
compose.yaml).
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

RUN useradd --create-home --uid 1000 appuser
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

USER appuser
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

3b. frontend/ — the static site baked into the nginx image

~/safer-shop/frontend/html/index.html:

```
~/safer-shop/frontend/html/index.html
```

~/safer-shop/frontend/Dockerfile — note it copies **only** the static content; the proxy config arrives at deploy time as a Swarm config:

```
FROM nginx:1.27-alpine
```

```
# Only the static site is baked in. nginx.conf is injected by the stack as a
# Docker config so you can change proxy behaviour WITHOUT rebuilding this
image.
```

```
COPY html/ /usr/share/nginx/html/
```

```
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
```

```
    CMD wget -q -O- http://localhost/ >/dev/null || exit 1
```

4. Build the images (Swarm will NOT do this for you)

`docker stack deploy` ignores every `build:` key it finds. On this single-node swarm the scheduler and the build host are the same machine, so a locally-built, locally-tagged image is immediately usable — no registry needed (your Step-1 choice).

```
docker build -t safa/api:1.0 ~/safa-shop/backend
```

```
[+] Building 41.9s (12/12) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 513B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.12-slim 3.6s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                       0.0s
=> [1/7] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f 3.7s
=> => resolve docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f 0.0s
=> => sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca 249B / 249B 0.5s
=> => sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf0 12.12MB / 12.12MB 3.0s
=> => sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70 1.29MB / 1.29MB 3.4s
=> => extracting sha256:3764a9a7d1e8a98213ab2201a41dbf3da9c60240ea35a5ea96ab994555070a70 0.1s
=> => extracting sha256:b0dc7f87bef15c2536cb182d6c48b52f90a0522884ec9eb222f833fa97ffebf0 0.2s
=> => extracting sha256:06ad939ed42b51caafb25b14810875d14e0ee0c441d3e2e5e46475fb8c150bca 0.0s
=> [internal] load build context                                  0.0s
=> => transferring context: 2.99kB                                    0.0s
=> [2/7] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* 9.4s
=> [3/7] RUN useradd --create-home --uid 1000 appuser             0.2s
=> [4/7] WORKDIR /app                                             0.0s
=> [5/7] COPY requirements.txt .                                  0.0s
=> [6/7] RUN pip install --no-cache-dir -r requirements.txt      22.9s
=> [7/7] COPY app/ ./app/                                         0.0s
=> exporting to image                                             2.0s
=> => exporting layers                                              1.6s
=> => exporting manifest sha256:754f598f681c3faea126f55487ee75a65572f222f7d9ac7d26cb0747a4daef1e 0.0s
=> => exporting config sha256:88ec9ae50b0198a5494f0d5b1da18b35191224163a0a3ed514fb42e65a43f51b 0.0s
=> => exporting attestation manifest sha256:28cb4dc9288fb0ef3b18c782df939fe9dd902145c7e75c4e27703d2cd39eacf7 0.0s
=> => exporting manifest list sha256:b87ac7617ec68307d10d7608688a3ec65302bc81fd38d766493416615c816e65 0.0s
=> => naming to docker.io/safa/api:1.0                             0.0s
=> => unpacking to docker.io/safa/api:1.0                          0.4s
```

```
docker build -t safa/nginx:1.0 ~/safa-shop/frontend
```

```
[+] Building 8.9s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 369B
=> [internal] load metadata for docker.io/library/nginx:1.27-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 294B
=> [1/2] FROM docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
=> => resolve docker.io/library/nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
=> => sha256:39c2ddfd6010082a4a646e7ca44e95aca9bf3eaeabc00f17f7ccc2954004f2a7d 15.52MB / 15.52MB
=> => sha256:34a64644b756511a2e217f0508e11d1a572085d66cd6dc9a555a082ad49a3102 1.40kB / 1.40kB
=> => sha256:81bd8ed7ec6789b0cb7f1b47ee731c522f6dba83201ec73cd6bca1350f582948 402B / 402B
=> => sha256:197eb75867ef4fcedcd4724f17b0972ab0489436860a594a9445f8eaff8155053 1.21kB / 1.21kB
=> => sha256:d7e5070240863957ebbb0b5a44a5729963c3462666baa2947d00628cb5f2d5773 955B / 955B
=> => sha256:b464cfd2a6319875aeb27359ec549790ce14d8214fcb16ef915e4530e5ed235 629B / 629B
=> => sha256:61ca4f733c802afd9e05a32f0de0361b6d713b8b53292dc15fb093229f648674 1.79MB / 1.79MB
=> => extracting sha256:61ca4f733c802afd9e05a32f0de0361b6d713b8b53292dc15fb093229f648674
=> => extracting sha256:b464cfd2a6319875aeb27359ec549790ce14d8214fcb16ef915e4530e5ed235
=> => extracting sha256:d7e5070240863957ebbb0b5a44a5729963c3462666baa2947d00628cb5f2d5773
=> => extracting sha256:81bd8ed7ec6789b0cb7f1b47ee731c522f6dba83201ec73cd6bca1350f582948
=> => extracting sha256:197eb75867ef4fcedcd4724f17b0972ab0489436860a594a9445f8eaff8155053
=> => extracting sha256:34a64644b756511a2e217f0508e11d1a572085d66cd6dc9a555a082ad49a3102
=> => extracting sha256:39c2ddfd6010082a4a646e7ca44e95aca9bf3eaeabc00f17f7ccc2954004f2a7d
=> [2/2] COPY html/ /usr/share/nginx/html/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:ddc94f6dd00fd93bbb97c072301f5fe227733c1b58a71941e724d65ec7316386
=> => exporting config sha256:de2481ce5bc6d636573a0ddbb610a38e0299fc724192bf6c0f2534dfdc92e186
=> => exporting attestation manifest sha256:2e24fcc840570b1eef596cd87214bc2a7a80ebdcb2acd52273e1158ec6160e69
=> => exporting manifest list sha256:a3023a05dbc0bcd63bca7461a8c149ba70ee2ef3a70fa4e8c04e54c76ec41ae
=> => naming to docker.io/safa/nginx:1.0
=> => unpacking to docker.io/safa/nginx:1.0
```

Confirm both images exist locally:

```
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}'
| grep '^safa/'
```

```
ubuntu@swarm-01:~$ docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}' | grep '^safa/'
safa/nginx          1.0          a3023a05dbc0 73.6MB
safa/api            1.0          b87ac7617ec6 283MB
```

Multi-node caveat (why "local images only" is single-node-only): if you later `docker swarm join` a second VM, that node has no `safa/api:1.0` and any task scheduled there fails with `No such image`. The moment you go multi-node you must push images to a registry all nodes can pull from. On this one-VM lab, local tags are correct and sufficient.

5. Write the deployment files (`safa-swarm-deploy/`)

Every file below uses `version: "3.9"` — the schema `docker stack deploy` expects and the one that enables the `deploy:` keys Swarm actually acts on.

5a. `postgres/` — owns the three Postgres secrets + the init config

```
~/safa-swarm-deploy/postgres/init/001_init.sql:
```

```
CREATE TABLE IF NOT EXISTS products (
  id SERIAL PRIMARY KEY,
```

```

    name          TEXT NOT NULL,
    price_cents   INTEGER NOT NULL
);

INSERT INTO products (name, price_cents) VALUES
    ('Widget A', 1999),
    ('Widget B', 4999),
    ('Widget C', 999)
ON CONFLICT DO NOTHING;

```

~/safer-swarm-deploy/postgres/docker-compose.yaml :

```

services:
  postgres:
    image: postgres:16-alpine
    networks:
      - backend-net
    environment:
      # The official image reads these *_FILE vars and loads the secret
      contents.
      POSTGRES_USER_FILE: /run/secrets/postgres_user
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
      POSTGRES_DB_FILE: /run/secrets/postgres_db
    secrets:
      - postgres_user
      - postgres_password
      - postgres_db
    configs:
      # Deliver the init SQL as a config mounted into the init hook dir – no
      bind mount.
      - source: pg_init_sql
        target: /docker-entrypoint-initdb.d/001_init.sql
    volumes:
      - pg_data:/var/lib/postgresql/data      # node-local named volume (see
placement below)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $(cat /run/secrets/postgres_user)
-d $(cat /run/secrets/postgres_db)"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
    deploy:

```

```

    replicas: 1
    placement:
      constraints:
        - node.role == manager # pin the DB to this node so it always
finds pg_data
    restart_policy:
      condition: on-failure
      delay: 5s

# --- resources OWNED by the postgres service (declared here once for the
whole stack) ---
secrets:
  postgres_user:
    file: ./secrets/postgres_user.txt
  postgres_password:
    file: ./secrets/postgres_password.txt
  postgres_db:
    file: ./secrets/postgres_db.txt

configs:
  pg_init_sql:
    file: ./init/001_init.sql

networks:
  backend-net:
    driver: overlay
    attachable: true

volumes:
  pg_data:

```

Why pin Postgres to a node (`node.role == manager`): a named volume in Swarm is **local to the node that created it**. If Postgres were free to reschedule onto a different node after a restart, it would land next to an *empty* `pg_data` and re-run the init script. Pinning it guarantees it always reunites with its data. On a one-node swarm this is automatic, but writing the constraint now means the file is already correct when you add nodes.

Path resolution: `file:` paths (`./postgres/secrets/...`, `./postgres/init/...`) are written **relative to the deployment root**, because we run `docker stack deploy` from `~/safer-swarm-deploy/`. Always deploy from that root and these resolve correctly.

5b. redis/ — owns the Redis secret

~/safer-swarm-deploy/redis/docker-compose.yml:

```
services:
  redis:
    image: redis:7.4-alpine
    networks:
      - backend-net
    secrets:
      - redis_password
    # redis-server can't read a password file directly, so read it inline at
    # start.
    # $$ escapes Compose interpolation → the container runs a literal $(cat
    # ...).
    command: ["sh", "-c", "redis-server --requirepass \"$$$(cat
    /run/secrets/redis_password)\" --appendonly yes"]
    volumes:
      - redis_data:/data
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a \"$$$(cat
      /run/secrets/redis_password)\" --no-auth-warning ping | grep -q PONG"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure
        delay: 5s

secrets:
  redis_password:
    file: ./secrets/redis_password.txt

networks:
  backend-net:
    driver: overlay
    attachable: true

volumes:
  redis_data:
```

5c. backend/ — the api; CONSUMES secrets it does not own

~/safer-swarm-deploy/backend/docker-compose.yml:

```
services:
  api:
    image: safer/api:1.0          # built in Step 4; NOT built here
    networks:
      - frontend-net            # reachable by nginx
      - backend-net             # reachable to postgres / redis – the
                                # only bridging service
    environment:
      PYTHONUNBUFFERED: "1"
      LOG_LEVEL: info
      POSTGRES_HOST: postgres
      REDIS_HOST: redis
      # Point the app at the secret files. The secrets themselves are
      # declared
      # top-level in postgres/ and redis/ – this service only references
      # them.
      POSTGRES_USER_FILE: /run/secrets/postgres_user
      POSTGRES_PASSWORD_FILE: /run/secrets/postgres_password
      POSTGRES_DB_FILE: /run/secrets/postgres_db
      REDIS_PASSWORD_FILE: /run/secrets/redis_password
    secrets:
      # source = the api's OWN top-level secret (declared below); target =
      # the
      # filename under /run/secrets the app reads. Keeping the api's objects
      # separate lets this file deploy on its own, in any order.
      - source: api_postgres_user
        target: postgres_user
      - source: api_postgres_password
        target: postgres_password
      - source: api_postgres_db
        target: postgres_db
      - source: api_redis_password
        target: redis_password
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 30s
```

```

    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
        max_attempts: 0          # keep retrying – this is our "wait for
the DB" mechanism
        delay: 5s
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first      # bring a new task up BEFORE killing the
old one
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
        reservations:
          cpus: "0.25"
          memory: 128M

# --- the api's OWN secrets (its own copies of the creds it consumes) ---
secrets:
  api_postgres_user:
    file: ./secrets/postgres_user.txt
  api_postgres_password:
    file: ./secrets/postgres_password.txt
  api_postgres_db:
    file: ./secrets/postgres_db.txt
  api_redis_password:
    file: ./secrets/redis_password.txt

networks:
  frontend-net:
    driver: overlay
    attachable: true
  backend-net:
    driver: overlay
    attachable: true

```

Notice `backend/` now declares its **own top-level `secrets:` block** pointing at `backend/secrets/*.txt`. That is what makes `deploy.sh` able to `docker stack deploy -c backend/docker-compose.yaml safa-stack` on its own — the file is self-contained. The trade-off is that these copies must stay byte-for-byte identical to the originals in

`postgres/secrets/` and `redis/secrets/` (§6 copies them for you). Swarm materializes them as distinct objects — `safo-stack_api_postgres_user`, etc. — separate from Postgres's own `safo-stack_postgres_user`.

Alternative — reference them as `external` instead of copying. If you'd rather not keep a second copy of the credentials, rely on `deploy.sh` deploying `postgres/` and `redis/` before `backend/` (it already does, in that order), then point the api at the swarm objects those deploys created:

```
secrets:
  - source: postgres_user      # the swarm object, not a file on
    disk
    target: postgres_user
    # ...postgres_password, postgres_db, redis_password the same way
secrets:
  postgres_user:
    external: true
    name: safo-stack_postgres_user # the namespaced name the postgres
    deploy created
    # ...one entry per secret
```

No duplicated files, but `backend/` becomes deploy-order-dependent and hard-codes the `safo-stack_` stack prefix. Pick one model — own copies (default below) or external — not both.

5d. `nginx/` — the edge proxy; delivers `nginx.conf` as a Swarm config

`~/safo-swarm-deploy/nginx/config/nginx.conf`:

```
worker_processes auto;

events {
    worker_connections 1024;
}

http {
    include      mime.types;
    default_type application/octet-stream;

    # Docker's embedded DNS (always 127.0.0.11 inside a container on an
    overlay).
```

```

# We resolve 'api' through it at REQUEST time, not at startup.
resolver 127.0.0.11 valid=10s ipv6=off;

server {
    listen 80;
    listen [::]:80;

    location / {
        root    /usr/share/nginx/html;
        index   index.html;
    }

    location /api/ {
        # Putting the target in a variable forces nginx to resolve 'api'
        # request via the resolver above – so nginx BOOTS even if the
        # service isn't up yet, instead of dying with [emerg] host not
        # found.
        # (A bare `upstream { server api:8000; }` is resolved once at
        # config-
        # parse time and crashes nginx when api is momentarily
        # unresolvable –
        # exactly what a separate/detached deploy order triggers.)
        set $api_upstream api:8000;
        proxy_pass http://$api_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_connect_timeout 3s;
        proxy_read_timeout 10s;
    }
}

```

~/safer-swarm-deploy/nginx/docker-compose.yaml:

```

services:
  nginx:
    image: safer/nginx:1.0          # built in Step 4
    command: ["nginx", "-g", "daemon off;"]
    networks:
      - frontend-net              # ONLY this network – no path to
      postgres/redis

```

```

ports:
  - target: 80
    published: 8080
    protocol: tcp
    mode: ingress # routing mesh: any node answers :8080
configs:
  - source: nginx_conf
    target: /etc/nginx/nginx.conf
healthcheck:
  test: ["CMD-SHELL", "wget -q -O /dev/null http://127.0.0.1/ || exit
1"]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 10s

deploy:
  replicas: 2
  restart_policy:
    condition: any
  update_config:
    parallelism: 1
    order: start-first

configs:
  nginx_conf:
    file: ./config/nginx.conf

networks:
  frontend-net:
    driver: overlay
    attachable: true

```

No `depends_on` anywhere. Swarm ignores it. If nginx starts before api, requests to `/api/` return `502` until an api task is healthy — then they just start working. If api starts before Postgres, api crashes, `restart_policy` relaunches it, and it succeeds once Postgres accepts connections. Ordering is emergent, not declared. This is the single biggest mental shift from `LAB.md`.

6. Create the secret files

Secrets are the whole reason to run this on Swarm — treat their files carefully.

```
cd ~/safer-swarm-deploy

printf 'app_user'           > postgres/secrets/postgres_user.txt
printf 'r7$kX2m!qLpZ9vN'   > postgres/secrets/postgres_password.txt
printf 'appdb'             > postgres/secrets/postgres_db.txt
printf 't4Vb9!wQeR2sYcJ'   > redis/secrets/redis_password.txt

# The api keeps its OWN copies (see §5c) so backend/ deploys independently.
# Copy — never re-type — so they stay byte-for-byte identical to the
# originals.
cp postgres/secrets/postgres_user.txt      backend/secrets/postgres_user.txt
cp postgres/secrets/postgres_password.txt  backend/secrets/postgres_password.txt
cp postgres/secrets/postgres_db.txt        backend/secrets/postgres_db.txt
cp redis/secrets/redis_password.txt        backend/secrets/redis_password.txt
```

Use `printf`, not `echo`. `echo` appends a newline; that newline becomes *part of the password*. Postgres would then be created with a password ending in `\n`, and your api — reading the same file — would match it, so it "works"... until some other client trims the newline and gets `authentication failed`. `printf` writes exactly the bytes you see. (Our `read_secret()` also `.strip()`s as a belt-and-braces defence.)

Lock the files down and keep them out of git:

```
chmod 600 postgres/secrets/*.txt redis/secrets/*.txt backend/secrets/*.txt
```

```
~/safer-swarm-deploy/.gitignore:
```

```
# Never commit real secret material.
**/secrets/*.txt
```

Verify the bytes are exactly right (no stray newline — note the count):

```
wc -c postgres/secrets/postgres_password.txt
```

```
ubuntu@swarm-01:~/safer-swarm-deploy$ wc -c postgres/secrets/postgres_password.txt
15 postgres/secrets/postgres_password.txt
ubuntu@swarm-01:~/safer-swarm-deploy$
```

15 characters, no 16th newline byte. Good.

7. Validate the merged stack before deploying

`docker stack deploy` has no `--dry-run`, but `docker stack config` renders the exact merged, interpolated result the way Swarm will load it — the pre-flight check you run after every edit.

```
cd ~/safer-swarm-deploy

docker stack config \
  -c postgres/docker-compose.yaml \
  -c redis/docker-compose.yaml \
  -c backend/docker-compose.yaml \
  -c nginx/docker-compose.yaml \
  > /dev/null && echo "merge + syntax OK"
```

```
ubuntu@swarm-01:~/safer-swarm-deploy$ docker stack config \
  -c postgres/docker-compose.yaml \
  -c redis/docker-compose.yaml \
  -c backend/docker-compose.yaml \
  -c nginx/docker-compose.yaml \
  > /dev/null && echo "merge + syntax OK"
merge + syntax OK
ubuntu@swarm-01:~/safer-swarm-deploy$
```

If a secret file path is wrong, or a service references an undeclared secret, it fails **here** instead of leaving a half-deployed stack:

```
# example failure if you fat-finger a path:
services.postgres.secrets.postgres_user: undefined secret "postgres_user"
```

If your Docker build predates `docker stack config`, substitute `docker compose -f postgres/... -f redis/... -f backend/... -f nginx/... config` — it validates the same merge (it may warn that `version` is obsolete; harmless for stack files).

8. Deploy the stack

One command, all four files, one stack named `safa-stack`. Save it as `deploy.sh` so nobody deploys a partial set by hand:

```
~/safa-swarm-deploy/deploy.sh:
```

```
#!/usr/bin/env bash
set -euo pipefail
docker stack deploy --detach=true -c postgres/docker-compose.yaml safa-stack
docker stack deploy --detach=true -c redis/docker-compose.yaml safa-stack
docker stack deploy --detach=true -c backend/docker-compose.yaml safa-stack
docker stack deploy --detach=true -c nginx/docker-compose.yaml safa-stack
```

```
cd ~/safa-swarm-deploy/
chmod +x ~/safa-swarm-deploy/deploy.sh
bash deploy.sh
```

```
Creating network safa-stack_backend-net
Creating secret safa-stack_postgres_user
Creating secret safa-stack_postgres_password
Creating secret safa-stack_postgres_db
Creating config safa-stack_pg_init_sql
Creating service safa-stack_postgres
Creating secret safa-stack_redis_password
Creating service safa-stack_redis
Creating network safa-stack_frontend-net
Creating secret safa-stack_api_postgres_user
Creating secret safa-stack_api_postgres_password
Creating secret safa-stack_api_postgres_db
Creating secret safa-stack_api_redis_password
Creating service safa-stack_api
Creating config safa-stack_nginx_conf
Creating service safa-stack_nginx
```

Everything is prefixed with the stack name `safa-stack_` — that namespacing is how Swarm keeps stacks from colliding on one cluster.

9. Watch the stack converge

Replicas do not appear instantly. Swarm pulls/starts tasks and gates them on healthchecks; watch the **REPLICAS** column climb to its target.

```
docker stack services safa-stack
```

First look — api still starting (it's crash-retrying until Postgres is ready):

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
bpezjcr5g8rq	safa-stack_api	replicated	0/3	safa/api:1.0	
hcsu95n29h54	safa-stack_nginx	replicated	0/2	safa/nginx:1.0	*:8080->80/tcp
tcaug224fh5o	safa-stack_postgres	replicated	1/1	postgres:16-alpine	
twyvps19fmgt	safa-stack_redis	replicated	1/1	redis:7.4-alpine	

Give it ~30–40s (the api **start_period**) and look again:

```
docker stack services safa-stack
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
lwo5b20bv64u	safa-stack_api	replicated	3/3	safa/api:1.0	
ydxit8vki1jj	safa-stack_nginx	replicated	2/2	safa/nginx:1.0	*:8080->80/tcp
0qyror9a78c9	safa-stack_postgres	replicated	1/1	postgres:16-alpine	
72cbfnueo2fi	safa-stack_redis	replicated	1/1	redis:7.4-alpine	

3/3 on api = all three replicas are up **and** passing their healthcheck. See where each task actually runs:

```
docker stack ps safa-stack --no-trunc
```

All tasks land on **safa-vm** — the only node. If you saw an api task with a **CURRENT STATE** of **Running** preceded by earlier **Failed** / **Shutdown** entries, that's the crash-retry-until-Postgres-ready loop doing its job; it's expected, not an error.

Confirm the secrets and configs Swarm materialized:

```
docker secret ls
```

ID	NAME	DRIVER	CREATED	UPDATED
krjr33lojl1bwgrhkq0s69984	safa-stack_api_postgres_db		7 minutes ago	7 minutes ago
a8lb71h6lv98zav0jenzymot4	safa-stack_api_postgres_password		7 minutes ago	7 minutes ago
qo6rfvtgl1qd2ov4qdrz40jfrk	safa-stack_api_postgres_user		7 minutes ago	7 minutes ago
vqm1o987s05w9qoa8hkipe1zh	safa-stack_api_redis_password		7 minutes ago	7 minutes ago
tuu0l2aqyjomsdvglxcb8mqq0	safa-stack_postgres_db		7 minutes ago	7 minutes ago
vzaxka7furxumfkr7c62olsk	safa-stack_postgres_password		7 minutes ago	7 minutes ago
ex1brxtld0naqemdbh338klzi	safa-stack_postgres_user		7 minutes ago	7 minutes ago
4xic6b5hsbbffjvy7cije9p6s	safa-stack_redis_password		7 minutes ago	7 minutes ago

The api's own `safa-stack_api_*` secrets sit alongside Postgres's and Redis's. Same credential values, distinct swarm objects — that's the price of a `backend/` that deploys on its own.

```
docker config ls
```

ID	NAME	CREATED	UPDATED
pdramlyyajed6d1vudvlaqdqj	safa-stack_nginx_conf	2 minutes ago	2 minutes ago
1x6gaqqrgrpnd7wzzuu41m13hx	safa-stack_pg_init_sql	8 minutes ago	8 minutes ago

10. Test the application through the routing mesh

Port `8080` is published in `ingress` mode, so the request enters the routing mesh and gets balanced to an nginx task.

```
curl -s http://localhost:8080/ | grep '<h1>'
```

```
<h1>Reverse proxy is up (Docker Swarm).</h1>
```

First product hit — served from Postgres (cache miss):

```
curl -s http://localhost:8080/api/products/1
```

```
{"id":1,"name":"Widget A","price_cents":1999,"source":"postgres"}
```

Second hit within 60s — served from Redis (cache hit), proving the api reached *both* backend services:

```
curl -s http://localhost:8080/api/products/1
```

```
{"id":1,"name":"Widget A","price_cents":1999,"source":"cache"}
```

`"source":"cache"` on the second request is your end-to-end proof: nginx → api (VIP-balanced across 3 replicas) → Postgres (first call) → Redis (second call), all over overlay networks, all using file-based secrets.

11. Reading logs across replicas

`docker service logs` aggregates **all replicas** of a service — no per-container hunting.

```
docker service logs --tail 5 safa-stack_api
```

```
safo-stack_api.1.plq2@safa-vm | INFO:      Application startup complete.
safo-stack_api.2.p3q4@safa-vm | INFO:      Application startup complete.
safo-stack_api.3.p5q6@safa-vm | INFO:      172.18.0.7:0 - "GET
/api/products/1 HTTP/1.0" 200 OK
safo-stack_api.1.plq2@safa-vm | INFO:      172.18.0.7:0 - "GET /health
HTTP/1.0" 200 OK
safo-stack_api.2.p3q4@safa-vm | INFO:      172.18.0.7:0 - "GET /health
HTTP/1.0" 200 OK
```

The `<service>.<replica#>.<taskID>@<node>` prefix tells you exactly which replica emitted each line. Follow live with `-f`:

```
docker service logs -f safa-stack_api
```

12. Run a one-off command inside a task

`docker service` has no `exec`. Find the concrete container of a task, then `docker exec` it. Verify the seeded data:

```
PG=$(docker ps --filter name=safa-stack_postgres -q)
docker exec "$PG" psql -U app_user -d appdb -c "SELECT * FROM products;"
```

```
id |   name   | price_cents
----+-----+-----
 1 | Widget A |         1999
 2 | Widget B |         4999
 3 | Widget C |          999
(3 rows)
```